

BALENISA SYSTEM DESIGN & ARCHITECTURE PLAN

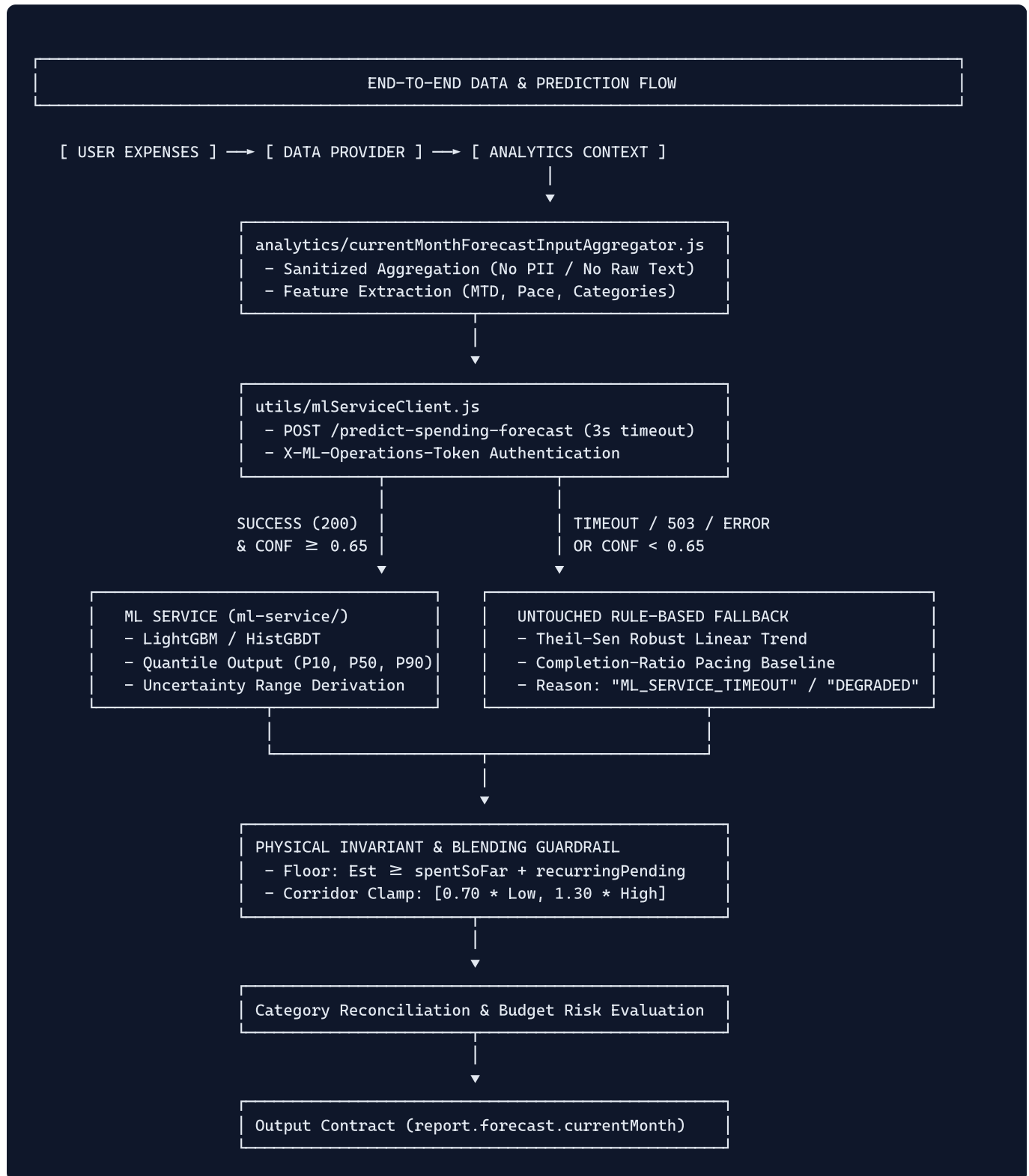
Spending Forecast ML Engine

Module-by-Module Technical Plan, Synthetic Data Pipeline, Model Training, & Zero-Touch Fallback Architecture

TARGET FEATURE	MODEL ARCHITECTURE	RELIABILITY STRATEGY	RULE-ENGINE POLICY
Spending This Month Nowcast	Quantile HistGBDT (P10, P50, P90)	Deterministic Theil-Sen Fallback	100% Preserved / Untouched

Core Architectural Invariant: The existing Node.js rule-based analytics engine is preserved completely intact. All new machine learning components are built as an isolated service in `m1-service/`. The rule-based engine serves as the guaranteed, zero-latency fallback whenever ML predictions are offline, timed out, or low confidence (<0.65).

1. System Architecture Overview



2. Module-by-Module Technical Specification

Module 1: Synthetic Spending Trajectory Generator

ML-SERVICE/TRAINING

File: ml-service/training/generate_forecast_dataset.py

Purpose: Generates a high-fidelity synthetic population of 10,000+ user-month spending series modeling realistic consumer finance personas to pre-train and calibrate the model without touching or needing live user production data.

Key Capabilities & Features:

- **Persona Modeling:** Models distinct wage schedules (1st of month, bi-weekly, variable freelancing), fixed bill clumping (rent/utilities on days 1–5), weekday commuting, weekend leisure multipliers (1.5x–2.2x), and Poisson-distributed discretionary micro-spending.
- **Sliding Observation Cut-Offs:** For each simulated month, extracts intra-month snapshots at cut-off days $d \in [1, 3, 5, 8, 12, 15, 18, 22, 25, 28]$.
- **Feature Extraction:** Extracts strictly $t \leq d$ observations: elapsedDay, progressRatio, spentSoFar, dailyVelocity, categoryShares, trailing3MonthMedian, recurringCommitted, recurringSpentSoFar.
- **Target Variable:** $y = \text{ActualRemainingSpend} = \text{TotalMonthEndSpend} - \text{spentSoFar}$.
- **Output Artifact:** ml-service/training/dataset/forecast_training_dataset.csv.

Module 2: Quantile GBDT Training Pipeline

ML-SERVICE/TRAINING

File: ml-service/training/train_spend_forecaster.py

Purpose: Trains an ensemble of three Gradient Boosted Decision Tree models using Pinball/Quantile Loss to produce point predictions along with asymmetric uncertainty bounds.

Technical Design & Implementation:

- **Model Architecture:** HistGradientBoostingRegressor (from standard scikit-learn, fully compatible with existing requirements.txt).
- **Loss Function:** Pinball Loss for $\alpha \in [0.10, 0.50, 0.90]$:

$$\mathcal{L}_{\alpha}(y, \hat{y}) = \max(\alpha(y - \hat{y}), (1 - \alpha)(\hat{y} - y))$$

- **Cross-Validation:** Group Time-Series CV grouped by userId to ensure zero data leakage across user identities or future time horizons.
- **Artifact Serialization:** Saves forecast_model_p50.pkl, forecast_model_p10.pkl, forecast_model_p90.pkl, forecast_scaler.pkl, and metadata.json inside ml-service/training/models/spend_forecast/.

Module 3: In-Memory Inference Engine & Confidence Estimator

ML-SERVICE/INFERENCE

File: ml-service/inference/spend_forecaster.py**Purpose:** Loads the model bundle into memory, performs ultra-fast inference (<2ms), derives dynamic confidence scores, and enforces physical invariant constraints.**Execution Logic:**

- **Point Prediction:** $\text{predictedTotal} = \text{spentSoFar} + \text{recurringPending} + \hat{y}_{P50}$.
- **Dynamic Confidence Score:** Evaluated based on quantile spread relative to total spend:

$$\text{confidenceScore} = \text{clamp}\left(1.0 - \frac{\hat{y}_{P90} - \hat{y}_{P10}}{\max(\text{predictedTotal}, 1.0)}, 0.0, 1.0\right)$$

- **Monotonic Floor Enforcement:** Guarantees $\text{predictedTotal} \geq \text{spentSoFar} + \text{recurringPending}$.

Module 4: Operational FastAPI Endpoint

ML-SERVICE/APP.PY

File: ml-service/app.py

Purpose: Exposes the authenticated prediction route POST /predict-spending-forecast.

Security & Protocol:

- Protected by `_require_operations_token` using header X-ML-Operations-Token.
- Validated against `SpendingForecastRequest` schema (Pydantic).
- Fail-closed: Returns 503 if operations token is unconfigured, 401 if invalid.

Module 5: Backend Service Client Integration

BACKEND/UTILS

File: backend/utils/mlServiceClient.js

Purpose: Centralizes the HTTP request mechanism from Node.js to the ML service with bounded timeout and automatic token injection.

Contract: `requestSpendingForecast(features, { timeoutMs = 3000 })`

- Encapsulates Axios request with 3000ms hard timeout.
- Injects `mlOperationsHeaders()` byte-for-byte.
- Catches network failures, DNS errors, timeouts, and 5xx responses and returns `{ success: false, fallback: true, reason }` without throwing exceptions.

Module 6: Forecast Analyzer Resilience & Fallback Bridge

BACKEND/ANALYTICS

File: backend/analytics/analyzers/currentMonthForecastAnalyzer.js

Purpose: Coordinates ML predictions with the untouched deterministic Theil-Sen statistical rule engine.

Execution Strategy:

- Runs ML request asynchronously.
- **Fallback Triggers:** ML timeout, HTTP error, malformed response, confidence score < 0.65, or history < 3 months.
- **Fallback Action:** Executes the 100% preserved `fitRobustTrend()` calculation from `robustTrend.js` and sets `method = "CURRENT_MONTH_ROBUST_NOWCAST_V1"` and `modelMetadata.isFallback = true`.
- **Happy Path Action:** Applies confidence-weighted blending and category allocation with `method = "ML_REGRESSION_ENSEMBLE_V1"`.

3. Step-by-Step Staged Execution Plan

STAGE	TARGET FILES	DESCRIPTION	RISK LEVEL
Stage 1	ml-service/training/generate_forecast_dataset.py	Build synthetic dataset generator and generate 10,000 training user-month trajectories.	Zero (Isolated)
Stage 2	ml-service/training/train_spend_forecaster.py	Train Quantile GBDT models (P10, P50, P90) and save model bundle.	Zero (Isolated)
Stage 3	ml-service/inference/spend_forecaster.py	Implement inference engine, confidence scoring, and boundary validation.	Zero (Isolated)
Stage 4	ml-service/app.py	Mount POST /predict-spending-forecast with token authentication.	Low
Stage 5	ml-service/tests/test_spend_forecaster.py	Execute ML service unit tests and contract validation.	Zero (Test Only)
Stage 6	backend/utils/mlServiceClient.js	Add requestSpendingForecast client helper in Node.js backend.	Low (Additive)
Stage 7	backend/tests/analytics.currentMonthForecast.test.js	Verify fallback execution against offline and degraded ML scenarios.	Zero (Test Only)

Deliverable Status: Ready to execute Stage 1 in ml-service/.